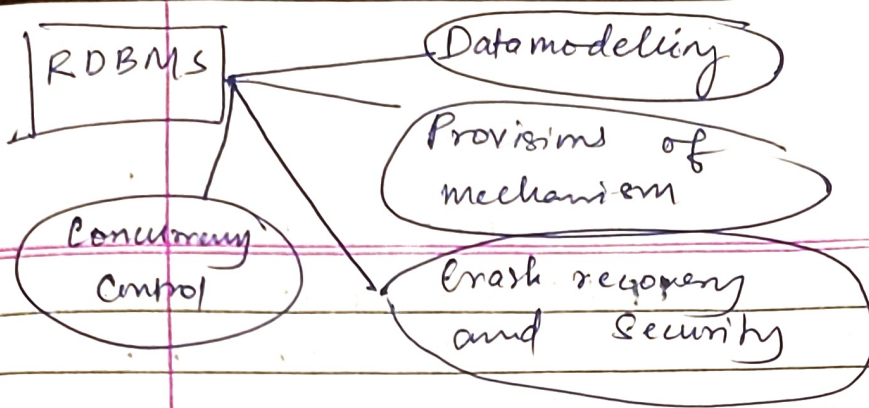




Date _____
Page _____

Vijay Rajesh R
DMS - Sem 5

⇒ Relational Algebra:

Select (σ): $\sigma_{\text{select condition}} (\text{Relation})$.

$\sigma_{\text{std_id} = "238410917"} (\text{student_table})$.

Project (π): $\pi_{\text{ID, Name, Salary}} (\text{INSTRUCTOR})$

Specific selection

Duplicates are removed.

* This will select only the columns ID, Name, Salary.

Union ($\cup$): $\pi_{\text{column}} (\text{Relation}_1) \cup \pi_{\text{column}} (\text{Relation}_2)$
U - operation

set Difference ($-$): $\pi_{\text{column}} (\text{Relation}_1) - \pi_{\text{column}} (\text{Relation}_2)$
R - S

Cartesian Product ($\times$):

$R \times S : \pi_{\text{column}} (\text{Condition} (R \times S))$.

1NF:

* Atomicity

2NF:

* 1NF, No partial dependency.

3NF:

1NF, 2NF and No transitive dependent

BCNF: 1NF 2NF 3NF and super keys on the left.

Aggregate functions :

1. COUNT
2. SUM
3. AVG
4. MIN
5. MAX

DCL: Access ^{Date} ^{Page} manipulation.
GRANT, REVOKE.

DDL: Data Definitive Language.
Create, Drop, TRUNCATE,
Alter.

DML: Data Manipulation Language.
Select, Update, Modify,
delete.

⇒ Architecture (DBMS) :

- Physical - storing in hardware, optimization.
- Logical - logical rules and schemas of DB.
- View level - Viewable portion to the users.

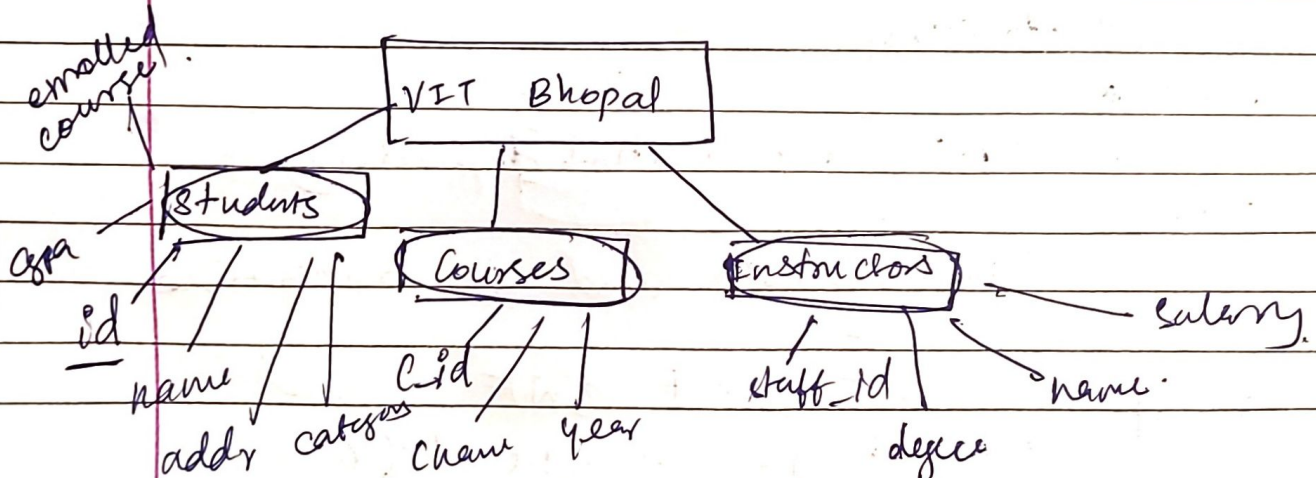
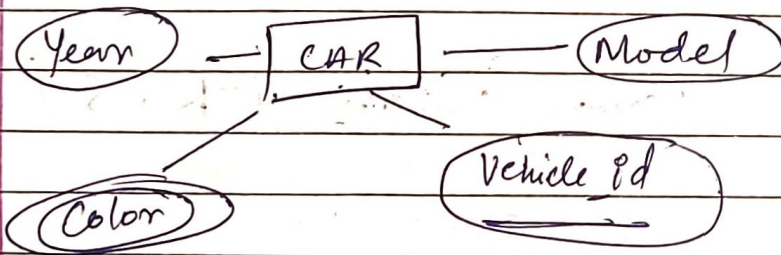
eg: A customer can see only their account whereas an admin can see all accounts.

⇒ Cardinality :

- Total no. of connections.
- (or) Possible no. of Occurrences.

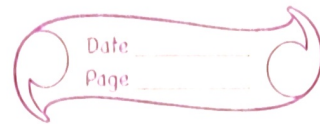
⇒ ER Diagram :

V.M.L

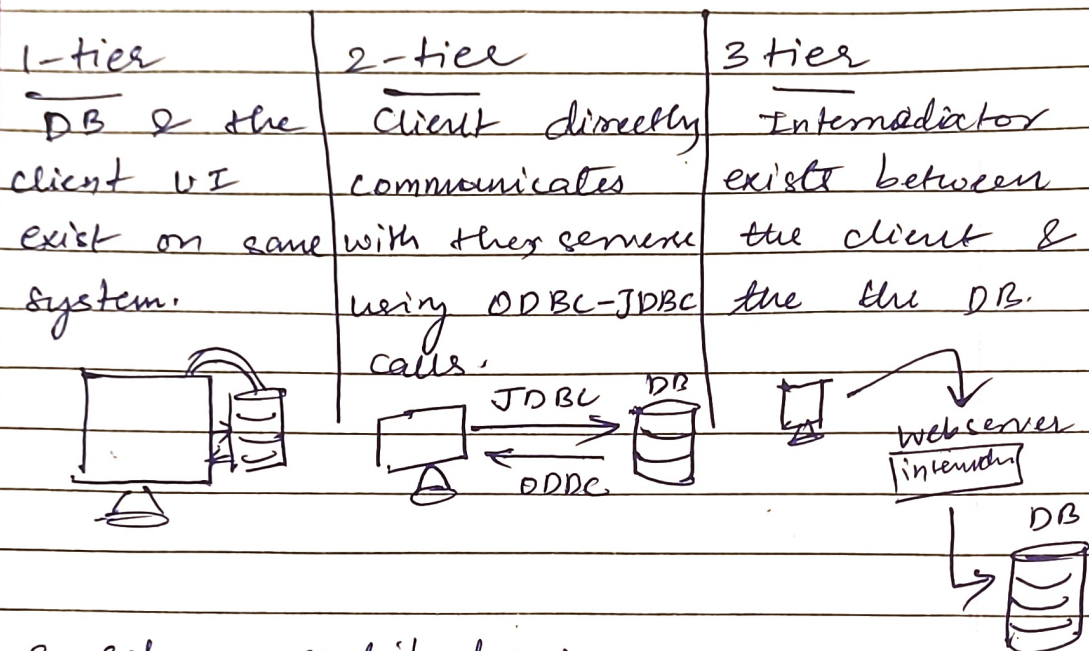


Transaction Control Language

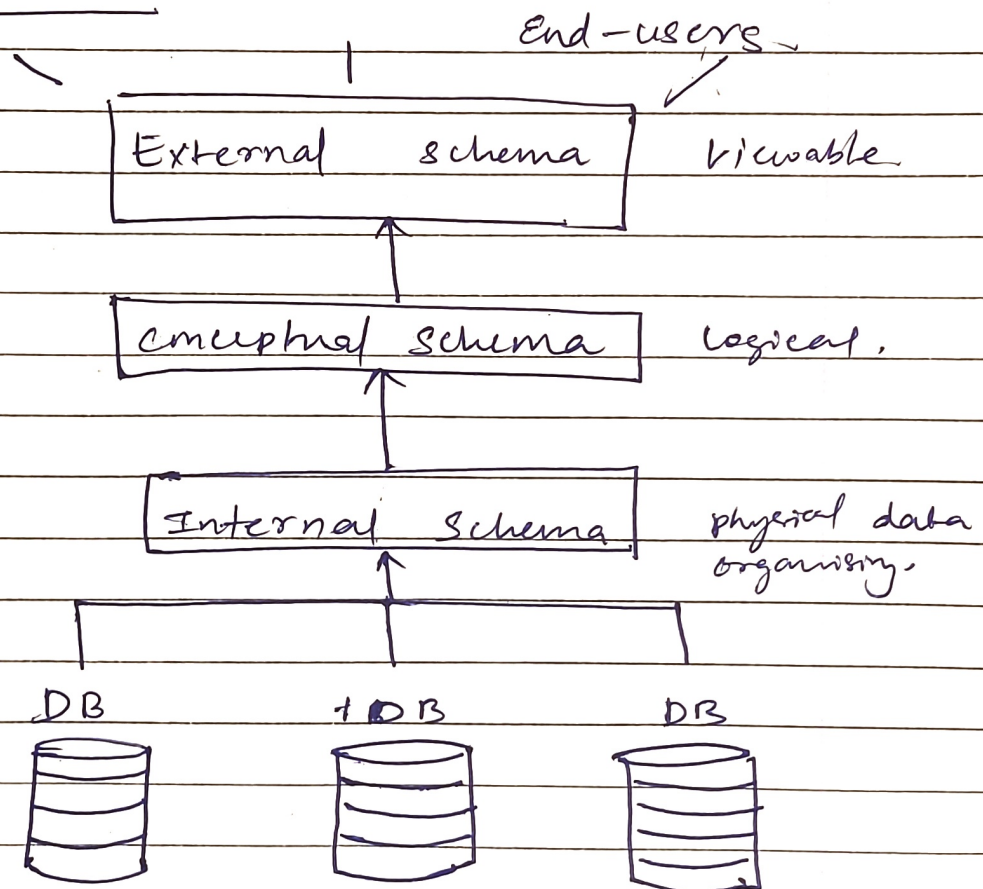
- Commit
- Rollback
- SAVEPOINT.



⇒ DBMS Architecture:



⇒ 3-schema architecture:



①: ER diagram, Architecture of DBMS, Relational databases, ~~DATE~~

②: Relational Algebra, UML, Joins, basic SQL, ~~Normal Forms~~, ~~codd rules~~, ~~keys~~

③: DML, DDL, TCL, Aggregate Functions, SQL queries, — order by, group by

Architecture of DBMS

Aggregati Functions

✓ ——— set based & tuple-based.
 Creating, drop, ✓ ——— Data, Definitive Language (DDL)
 alter, select, delete, ✓ ——— Data Manipulation Language (DML)
 ✓ ——— Data Control Language (DCL)

Q2. ~~Customers~~ Table Column

```
SELECT customers.customer_id, orders.product
FROM customer LEFT JOIN orders ON customer_id = customers.customer_id
```

1. Count
2. Sum ()
3. Avg ()
4. MAX ()
5. ~~for~~ MIN ()

SELECT ~~*~~ COUNT(*) FROM
 ,, SUM (values) ,,

DAL

DOL

DML

10 CL

JOIN &

Data Entry Layer 15

SQL FUNCTIONS

Fundamental ops:

Select (σ)	$\cap$	Outer join
Project (π)	$=$	left outer ($\Join$)
Union ($\cup$)	$\Join$	Right outer ($\Join$)
Set diff ($-$)	$(\div)$	$\Join$
Cartesian Product ($\times$)		

Select (σ):

$\sigma_{(\text{select condition})}(\text{Relation})$

Eg: $\sigma_{(\text{std_id} = "23BA110917")}(Student_table)$

* Comparison operators:-

$\neq$, $<$, $=$, Δ , $\leq$, $>$ and $\geq$.

* connection of multiples can be done by connectives: ($\wedge$, $\vee$, and $\neg$)

Sat-maths class / revise
Sun-

Aiml - Test Tuesday

Monday-

Wednesday-

Thursday &

Friday.

Architecture

1) Physical - Physical, database
indexing, optimisations
indexing

2) Logical - Struct: set
of rules & definitions
- Structure of DB.

3) View level - US
bank employee sees both
all the accounts where
customer sees only
Accounts.

Select (σ):

①

σ (dept_name = "Finance")

Reference: select show ^{all} instructors from
finance department.

②

Select (σ): Find instructors with salary > 87000

σ (salary > 87000) (INSTRUCTOR)

③

Find instructors working in "Finance" department
and salary > 87000.

σ (Dept_id = "Finance" ∧ salary > 87000) (INSTRUCTOR)

Project (π) :-

× ~~specific~~ selection specific selection

× Duplicates are removed.

Eg,

π (ID, Name, Salary) (INSTRUCTOR)

this will give only the columns of ID, Name, Salary.
Rest of the columns will be removed.

①

Find names of all instructors in CS dept.

Sol

π (name, ~~dept_name~~) (INSTRUCTOR)

②

Find name of all instructors in CS

π name (σ (dept_id = "CS") (INSTRUCTOR))

→ Union ($\cup$): no. of columns should be same.
 → Any Binary operator.

PAGE NO. _____
 DATE _____

- Duplicate rows are eliminated.
- Certain conditions should be met such as same no. of attributes on both sides such that $\cup$ op can be done.

Syntax: $\pi_{\text{column}}(\text{Relation}_1) \cup \pi_{\text{column}}(\text{Relation}_2)$

Eg.:

A	characters	A $\cup$ B
A	H	A
B	V	B
C	W	C
E	\$	E
F	X	F
	Y	W
	Z	\$
		X
		Y
		Z

duplicates are eliminated

(DB)

Q Find set of all courses taught in Fall 2009 Spring 2010 or both.

→ Fall 2009

$\pi_{\text{course}}(\sigma_{\text{semester} = \text{"Fall"} \wedge \text{year} = 2009}(\text{SECTION}))$

→ Spring 2010

$\pi_{\text{course}}(\sigma_{\text{semester} = \text{"Spring"} \wedge \text{year} = 2010}(\text{SECTION}))$

→ Both Fall 2009 and Spring 2010.

$\pi_{\text{course}}(\sigma_{\text{semester} = \text{"Spring"} \wedge \text{year} = 2010}(\text{SECTION}))$

$\cup \pi_{\text{course}}(\sigma_{\text{semester} = \text{"Fall"} \wedge \text{year} = 2009}(\text{SECTION}))$

Set Difference (-) : (Binary)

$R - S$ (tuples in R but not in S)

$S - R$ (tuples in S but not in R)

eg: $R - S$

R	Alphabets	S	Characters
	A		B
	B		C
	C		\$
	D		@
	E		E D
	F		

$R - S$
A
B
C
D
E
F

$\pi_{(us_name)} (depositors) - \pi_{(us_name)} (Borrowers)$

R-S

S-R

1-N: Atomicity columns -

Remove repeating columns.

2-N:

= 1NF + no partial dependencies

3-NF:

1NF + 2NF + No transitive dependency

BCNF:

1NF + 2NF + 3NF + 1 superkey
per table,

Cartesian product:

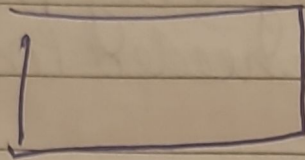
Product. Every tuple of R_1 with every tuple of R_2 .

→ $R_1 \times R_2 =$ All possible pairing

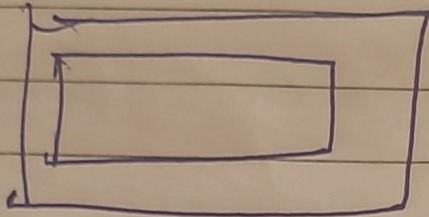
→ It is binary ops.

is
(tablename.column name.)

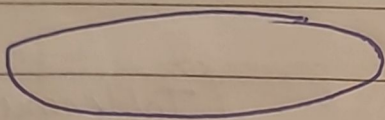
ER diagrams (symbols)



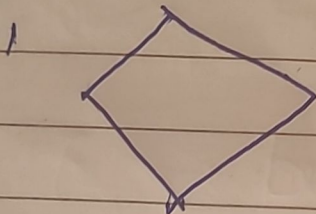
= Strong entity



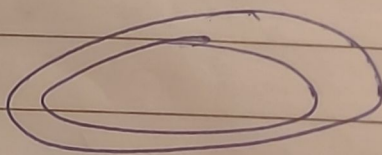
= Weak entity



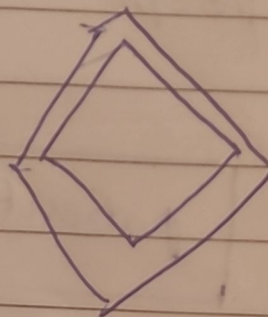
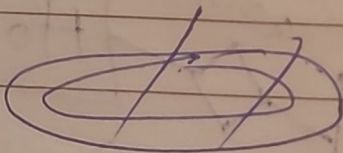
= Attributes



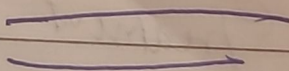
= Relationship set



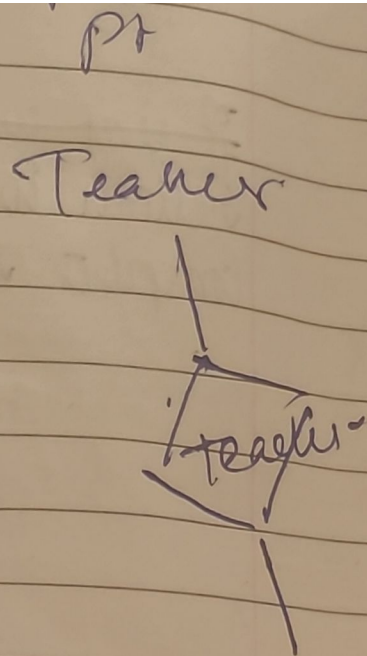
Multivalued attrib.
Relationship type



Relationship
type



Full participation



DELSQL

3contrains.sql

1StudentsList.sql

ELSQL > ...

▷Run | +Tab | JSON | Active Connection

1 SELECT customers.customer_id,orders.product

2
3 FROM customer LEFT JOIN orders ON orders.customer_id = customers.customer_id;

4
5 ▷Run | +Tab | JSON

6 SELECT students.name,classes.name,students.class_id FROM students

7 RIGHT JOIN classes ON classes.class_id = students.class_id;

8
9 ▷Run | +Tab | JSON

10 SELECT employees.emp_name,employees.department_id,departments.department_name

11 FROM employees FULL JOIN departments ON departments.department_id=employees.department_id;